Лабораторная работа № 2.

Установка Docker и управление контейнерами

Тема: Установка Docker Engine на Debian 13, управление docker.service и жизненным циклом контейнеров

Цель: Установить Docker Engine из официального APT-репозитория, освоить systemd и команды управления контейнерами, научиться безопасно диагностировать и устранять ошибки.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №2. Установка Docker Engine и управление контейнерами. Продолжительность: 4 академических часа (180 минут). Система: Debian 13 (Trixie), установленная в отдельной учебной виртуальной машине или на учебном сервере, на котором разрешена установка системных пакетов.

Внимание: лабораторная включает команды управления системной службой и установку пакетов. Выполняйте их только на собственной учебной ВМ. Перед экспериментом рекомендуется сделать снимок ВМ. На общем сервере останавливать Docker запрещено.

2. Тема, цель и результаты

Тема: установка Docker Engine из официального APT-репозитория, проверка docker.service, запуск, диагностика, остановка, повторный старт и удаление контейнеров.

- Установить Docker Engine корректным способом для Debian 13.
- Объяснять назначение пакетов docker-ce, docker-ce-cli и containerd.io.
- Работать с Docker Daemon через systemctl и journalctl.
- Управлять контейнерами: docker run, ps, stop, start, restart, exec, logs, inspect, rm.
- Выявлять и устранять безопасные типовые ошибки.

3. Оборудование и исходные условия

- Debian 13 с доступом к интернету и APT-репозиториям; sudo-права.
- VirtualBox или отдельный учебный компьютер; терминал Debian.
- Доступ к download.docker.com по HTTPS и к реестру для образа hello-world и nginx:stable.
- Утилита curl (будет установлена в ходе работы).
- Команды выполняются в терминале Debian; Windows используется только как хост при VirtualBox.

Учебные значения

Объект	Значение	Примечание
ОС	Debian 13 / trixie	Проверить в /etc/os-release
Сервис Docker	docker.service	systemd
Веб-контейнер	lab2-web	Имя контейнера
Порт веб-контейнера	80/tcp	Внутри контейнера
Порт Debian-хоста	8085/tcp	Только 127.0.0.1

Объект	Значение	Примечание
Второй контейнер	lab2-test	Для проверки повторного запуска

4. Подготовка системы и безопасность

Если первая лабораторная проводилась на ВМ с уже установленным Docker, используйте отдельный снимок или чистую учебную ВМ для отработки установки. Не удаляйте рабочую установку Docker ради повторения лабораторной.

Этап 1. Уточнение операционной системы

```
Определить Debian и архитектуру
```

```
cat /etc/os-release
dpkg --print-architecture
whoami
sudo -v
```

Команда cat читает сведения об ОС, dpkg показывает архитектуру пакетов, whoami — текущего пользователя, sudo -v подтверждает право административного доступа. Ожидается Debian GNU/Linux 13 с VERSION_CODENAME=trixie и успешный вызов sudo.

Этап 2. Проверка существующих пакетов

```
Проверка CLI и установленного Docker
```

```
command -v docker || true
dpkg -l docker.io docker-ce docker-ce-cli containerd runc 2>/dev/null || true
```

Команды не изменяют систему. Если docker-ce уже установлен, пропустите установку и переходите к этапу проверки службы. Если есть docker.io или действующие контейнеры — остановитесь и сообщите преподавателю; автоматическое удаление чужих пакетов запрещено.

5. Установка официального Docker Engine

Этап 3. Системные зависимости

```
Подготовка APT и HTTPS
```

```
sudo apt update
sudo apt install -y ca-certificates curl
```

apt update обновляет индексы пакетов; apt install устанавливает ca-certificates и curl. Параметр -y автоматически подтверждает вопросы пакетного менеджера. Критерий продолжения: обе команды завершаются успешно.

Этап 4. Ключ подписи Docker

```
Добавить официальный ключ
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Создаётся каталог /etc/apt/keyrings и файл docker.asc. Права чтения позволяют APT проверять подписи пакетов. Не исполняйте загруженный ключ как скрипт.

Проверить файл ключа

```
ls -l /etc/apt/keyrings/docker.asc
```

Ожидается существующий файл с правами на чтение. Если файл отсутствует, проверьте DNS, HTTPS и правильность URL.

Этап 5. Файл репозитория deb822

Создание /etc/apt/sources.list.d/docker.sources

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Команда tee создаёт системный файл. Конструкция $(...) вычисляет codename и архитектуру до записи; для Debian 13 ожидается Suites: trixie. Этот шаг не меняет существующие системные репозитории Debian.

Проверка созданной конфигурации

```
cat /etc/apt/sources.list.d/docker.sources
sudo apt update
apt-cache policy docker-ce
```

Ожидаются корректные поля источника и доступная версия docker-ce в apt-cache policy. Если apt сообщает об ошибках подписи, проверить путь Signed-By и файл docker.asc.

Этап 6. Установка пакетов

Установка официальных пакетов

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

docker-ce устанавливает Engine; docker-ce-cli — команду docker; containerd.io — runtime-компоненты; остальные пакеты добавляют сборку и Compose. Критерий продолжения: команда apt закончилась с кодом 0.

Проверка версий компонентов

```
docker --version
sudo docker version
sudo docker compose version
```

docker --version показывает версию клиента. docker version должен содержать данные Client и Server, когда служба работает. Docker Compose представлен отдельной командой docker compose version.

6. Управление службой Docker

Этап 7. Статус службы

```
Проверка systemd

sudo systemctl status docker --no-pager
sudo systemctl is-active docker
sudo systemctl is-enabled docker
```

systemctl status предоставляет подробное состояние; is-active сообщает active или другое состояние; is-enabled — настройки старта при загрузке. Если служба inactive, выполните sudo systemctl start docker.

Этап 8. Настройка автоматического запуска

```
Включить автозапуск сервиса

sudo systemctl enable docker
sudo systemctl is-enabled docker
```

Команда enable настраивает запуск службы при старте ОС. Ожидается enabled. Не следует путать это с автоматическим перезапуском отдельных контейнеров.

Этап 9. Безопасный эксперимент со службой

Эксперимент выполняется только на отдельной учебной ВМ, до запуска пользовательских контейнеров. Остановите docker.service, затем проверьте, возможны ли обращения к Docker. На системах с docker.socket активация по требованию может автоматически перезапустить службу при обращении к CLI; это нормальная особенность systemd, а не ошибка лабораторной.

```
Остановить службу и увидеть состояние

sudo systemctl stop docker.service docker.socket
sudo systemctl is-active docker.service
```

Ожидается inactive. Команда не удаляет установленные пакеты, контейнеры и образы. Для проверки daemon необязательно вызывать docker ps, если это может активировать сокет.

```
Восстановить работу

sudo systemctl start docker.socket docker.service
sudo systemctl is-active docker.service
sudo docker info
```

Ожидается active и успешное получение информации Docker. Эксперимент считается завершённым только после восстановления работоспособности.

```
Последние сообщения службы

sudo journalctl -u docker.service -n 30 --no-pager
```

journalctl читает журнал systemd; -u выбирает конкретную службу, -n ограничивает количество записей. В отчёте укажите найденные события остановки и запуска.

7. Проверка установки тестовым контейнером

Этап 10. Запуск hello-world

```
Тестовый контейнер

sudo docker run --rm hello-world
sudo docker images
```

run загружает образ при необходимости, создаёт контейнер и запускает программу. --rm удаляет контейнер после завершения. Ожидается текст Hello from Docker!, а образ hello-world должен остаться среди локальных образов.

8. Создание веб-контейнера и параметры запуска

Этап 11. Фоновый запуск Nginx

```
Запуск Nginx с localhost-привязкой

sudo docker run -d --name lab2-web -p 127.0.0.1:8085:80 nginx:stable
```

Флаг -d запускает в фоне, --name задаёт имя, -p 127.0.0.1:8085:80 публикует порт 80 контейнера на loopback-адресе Debian-хоста. Это ограничивает доступ к веб-серверу самой Debian-системой; с Windows адрес localhost не будет адресовать виртуальную машину.

```
Проверка доступности

sudo docker ps
curl -I http://127.0.0.1:8085
```

В docker ps ожидается контейнер lab2-web со статусом Up. В curl ожидается успешный HTTP-ответ от Nginx (обычно 200 OK). Если сервер не отвечает, проверьте STATUS и публикацию порта.

Разбор параметров docker run

Фрагмент	Назначение
-d	Запуск без привязки к терминалу
--name lab2-web	Имя экземпляра
-p 127.0.0.1:8085:80	Порт хоста 8085 → порт контейнера 80

Фрагмент	Назначение
nginx:stable	Имя и тег исходного образа

Этап 12. Инспектирование

```
Сведения о контейнере

sudo docker inspect lab2-web
sudo docker port lab2-web
```

inspect возвращает структурированные свойства контейнера; docker port — опубликованные порты. Найдите поля Name, State, Image, NetworkSettings и HostConfig.PortBindings. Критерий: порт 80/tcp связан с 127.0.0.1:8085.

Этап 13. Выполнение команды внутри

```
Команды внутри работающего контейнера

sudo docker exec lab2-web hostname
sudo docker exec lab2-web cat /etc/os-release
sudo docker exec -it lab2-web /bin/sh
```

exec не создаёт новый контейнер, а запускает команду в работающем. hostname должен вернуть имя контейнерного узла. После входа в /bin/sh выполните pwd и ls /etc/nginx, затем exit — веб-сервер при этом продолжит работать.

9. Жизненный цикл контейнеров

Этап 14. Остановка и запуск

```
Остановить и проверить

sudo docker stop lab2-web
sudo docker ps
sudo docker ps -a
```

После stop lab2-web исчезнет из docker ps, но останется в docker ps -a со статусом Exited. HTTP-запрос к 127.0.0.1:8085 больше не должен обрабатываться этим контейнером.

```
Повторный запуск

sudo docker start lab2-web
curl -I http://127.0.0.1:8085
```

start повторно запускает ранее созданный контейнер, сохраняя его имя и конфигурацию порта. Не требуется выполнять новый docker run. Ожидается снова успешный HTTP-ответ.

Этап 15. Перезапуск и журналы

```
Рестарт и логи

sudo docker restart lab2-web
```

```
sudo docker logs --tail 20 lab2-web
```

restart выполняет остановку и повторный запуск контейнера. logs выводит последние записи; предварительный HTTP-запрос увеличивает вероятность появления записи о запросе в журнале.

Этап 16. Создание без запуска

```
Создать контейнер и запустить отдельно

sudo docker create --name lab2-test nginx:stable
sudo docker ps -a
sudo docker start lab2-test
sudo docker ps
```

docker create создаёт контейнер в состоянии Created, но не запускает основной процесс. start запускает этот экземпляр. Второй Nginx не публикует порт на хосте, поэтому не конфликтует с lab2-web.

10. Диагностические эксперименты

Эксперимент А. Конфликт уникального имени

Намеренно повторите создание контейнера с именем lab2-web. Docker должен отказать, поскольку это имя уже занято. Ошибка безопасна и не изменяет работающий контейнер.

```
Повтор имени

sudo docker run -d --name lab2-web nginx:stable
```

Ожидается сообщение Conflict: имя контейнера уже используется. Для восстановления ничего удалять не нужно; используйте другое имя. Проверьте существующий экземпляр командой sudo docker ps -a.

Эксперимент Б. Конфликт опубликованного порта

Здесь проверяем занятие конкретного порта Debian-хоста. Создадим контейнер с отдельным именем и тем же host-портом. Docker может создать неработающий контейнер до сообщения об ошибке; поэтому предусмотрена адресная очистка.

```
Проверка конфликта порта

sudo docker run -d --name lab2-conflict -p 127.0.0.1:8085:80 nginx:stable
```

Ожидается отказ в публикации порта, если lab2-web уже удерживает 127.0.0.1:8085. Зафиксируйте текст ошибки, затем удалите только lab2-conflict.

```
Восстановление после эксперимента

sudo docker rm -f lab2-conflict
sudo docker ps
```

Параметр -f принудительно удаляет указанный учебный контейнер, если он был создан; используйте его только для lab2-conflict. Контейнер lab2-web должен продолжить работу.

Эксперимент B. Остановка и диагностика

```
Состояние и восстановление

sudo docker stop lab2-test
sudo docker ps -a
sudo docker logs --tail 10 lab2-test
sudo docker start lab2-test
```

Найдите изменение статуса и убедитесь, что запускается тот же экземпляр. Эксперимент обратим через docker start.

11. Итоговая проверка и очистка

```
Проверить оба учебных контейнера

sudo docker ps -a
sudo docker images
sudo docker port lab2-web
curl -I http://127.0.0.1:8085
```

Критерий: lab2-web работает и отвечает через 8085; lab2-test существует; nginx:stable и hello-world доступны локально. Неопределённые состояния зафиксируйте и объясните по логам.

```
Адресная очистка

sudo docker stop lab2-web lab2-test
sudo docker rm lab2-web lab2-test
sudo docker ps -a
sudo docker images
```

Эти команды удаляют только два контейнера данной лабораторной. Образы сохраняются. Не выполняйте docker system prune -a и не удаляйте /var/lib/docker — это может уничтожить другие учебные данные.

12. Индивидуальное задание

1. Создайте контейнер nginx:stable с именем student-nginx и публикацией порта 127.0.0.1:8092:80.
2. Получите снимок результата curl -I http://127.0.0.1:8092.
3. Докажите через inspect или port, как сопоставлены порты.
4. Остановите и повторно запустите контейнер с сохранением имени.
5. Соберите его логи после двух HTTP-запросов.
6. Удалите только student-nginx и проверьте, что образ nginx:stable остался.

13. Содержание отчёта

7. Титульный лист, цель, используемая ОС и архитектура.
8. Вывод /etc/os-release, docker version и docker compose version.
9. Содержимое файла docker.sources и подтверждение доступности docker-ce в APT.
10. Состояние docker.service до и после эксперимента с systemd.
11. Скриншот выполнения hello-world и списка образов.
12. Команда docker run для Nginx, вывод docker ps и проверка curl.

13. Фрагменты docker inspect / docker port и результат docker exec.
14. Статусы после stop и после start; диагностические ошибки конфликтного имени и порта.
15. Итоговый индивидуальный запуск, очистка и вывод своими словами.

14. Контрольные вопросы

16. Почему для Debian 13 рекомендуется использовать официальный репозиторий Docker?
17. Что хранится в /etc/apt/sources.list.d/docker.sources и зачем нужен ключ docker.asc?
18. В чём различие между docker-ce и docker-ce-cli?
19. Какая разница между службой docker.service и контейнером lab2-web?
20. Что меняет systemctl enable docker?
21. Почему доступ к Docker обычно требует sudo?
22. Чем docker create отличается от docker run?
23. Чем docker run отличается от docker start?
24. Для чего нужны -d, --name, -p и -it?
25. Почему нельзя дважды использовать одно имя контейнера?
26. Что означает публикация 127.0.0.1:8085:80?
27. Какие команды помогают найти причину остановки контейнера?
28. Что удаляется при docker rm, а что сохраняется?
29. Почему остановка Docker Daemon не эквивалентна docker stop NAME?

15. Критерии оценивания

Оценка по 100-балльной системе

Критерий	Баллы
Подготовка системы и корректная установка	20
Управление службой Docker и восстановление	15
Проверка hello-world	10
Запуск и проверка Nginx	15
Жизненный цикл, exec, inspect и логи	15
Диагностические эксперименты	10
Индивидуальное задание	10
Отчёт и ответы на вопросы	5
Итого	100

16. Официальные источники

- Docker Docs. Install Docker Engine on Debian: https://docs.docker.com/engine/install/debian/
- Docker Docs. Linux post-installation steps: https://docs.docker.com/engine/install/linux-postinstall/
- Docker Docs. docker container run reference: https://docs.docker.com/reference/cli/docker/container/run/